

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 1– Concept Mapping**

Course Code:	CSA03	Course Title:	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	Assessment:	Assessment Tool 1 – Concept Mapping
Weightage:	20 % of CIA	Total Marks:	100
CO1 Statement:	Basics, Data, Data object, Data structure, Abstract Data Types (ADT), realization of ADT in 'C', Primitive and non-primitive data structure, Linear and non-linear data structure Arrays & Recursion, Performance analysis and Measurement: Time and space analysis of algorithms, Average, best and worst-case analysis.		
Student Name:	G. Ram Charan	Reg. No.:	192525043
Section / Batch:	AI&ML 2 nd year 2025-2029	Date:	21-07-2025

Code of Conduct

I, G.Ram Charan(192525043) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

G.Ram Charan

Signature of the Student: _____

Rubrics

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are wellstructured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Annexure A – Concept Mapping

Questions:	EACH QUESTIONS CARRIES : 10 MARKS	Duration :ONE HOUR
-------------------	--	---------------------------

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Q3 E-commerce platform manages product storage, searching, and recommendation systems. Construct a concept map linking Linear Data Structures → Sequential Access → Limitations → Non-Linear Data Structures → Hierarchical/Network Relationships → Applications. Analyze the differences between linear and non-linear structures and demonstrate how each supports specific components of the system. Justify which structure is more efficient for handling complex applications.

Q4 A ride-sharing application must efficiently match users with nearby drivers in real time. Construct a concept map connecting Algorithm → Input Size → Time Complexity → Space Complexity → Efficiency → Performance Measurement. Illustrate how increasing input size impacts system performance and establish relationships between these concepts. Analyze how the choice of algorithm affects efficiency and justify the most suitable approach.

Q5 A mobile application operates under strict memory constraints while processing large datasets. Construct a concept map linking Memory Usage → Space Complexity → Auxiliary Space → Input Size → Algorithm Efficiency. Include examples of algorithms with varying space complexities and explain how memory limitations influence the selection of data structures and algorithms. Justify how efficient memory usage improves system performance.

Q6 A banking system performs operations such as deposits, withdrawals, and transaction tracking. Construct a concept map connecting ADT → Operations → Interface → Implementation → Data Structures. Include examples of banking operations and explain how ADTs separate logical design from implementation. Justify how this abstraction improves modularity, reusability, and maintainability of the system.

Q7 A file system contains nested folders and files organized hierarchically. Construct a concept map linking Recursion → Base Case → Recursive Case → Call Stack → Function Calls → Memory Usage. Analyze how recursion operates internally and explain the relationship between recursive calls and stack memory. Justify why recursion is suitable for solving hierarchical problems.

Q8 A program processes complex nested data structures and requires efficient computation. Construct a concept map connecting Recursion → Call Stack → Memory Usage → Iteration → Loop Control → Efficiency. Compare the two approaches in terms of execution flow, memory consumption, and performance. Analyze their advantages and limitations, and justify which approach is more efficient in different scenarios.

Q9 A music streaming application manages playlists where songs are added, removed, and played sequentially. Construct a concept map linking Array → Linked List → Stack → Queue → Operations (Insert/Delete/Traverse). Explain how each structure can be applied in playlist management and justify the most suitable structure based on performance and flexibility.

Q10 A social media platform represents users and their connections in a network. Construct a concept map connecting Tree → Hierarchical Data → Graph → Network Data → Traversal (DFS/BFS). Analyze how these structures model relationships and support efficient searching. Justify which structure is more suitable for representing social networks.

Q11 A system needs to search for specific records in a large dataset efficiently. Construct a concept map linking Searching Algorithm → Linear Search → Binary Search → Time Complexity → Efficiency. Include relationships that compare the number of operations and performance of each method. Analyze which algorithm is more efficient and justify your conclusion.

Q12 A ride-sharing application must manage user requests, driver allocation, and route mapping simultaneously. Construct a concept map connecting ADT → Linear Structures → Non-Linear Structures → Algorithms → Performance Analysis → Efficiency. Demonstrate how these concepts are integrated into a real-world system. Justify the role of each component in achieving efficient and scalable performance.

Q13 A library management system stores thousands of book records including title, author, and availability status. Construct a concept map linking Data → Data Structure → Organization → Access Methods → Efficiency. Include different data structures used for storing and retrieving records. Explain how proper organization improves search efficiency and system performance.

Q14 A ticket booking system supports operations such as booking, cancellation, and seat allocation. Construct a concept map connecting Abstract Data Type (ADT) → Operations → Implementation → Data Structures → Performance. Analyze how different implementations (array vs linked list) affect efficiency. Justify the best choice for real-time booking systems.

Q15 A weather monitoring system collects temperature, humidity, and wind data continuously. Construct a concept map linking Primitive Data Types → Non-Primitive Structures → Linear Structures → Non-Linear Structures → Applications. Include examples and explain how each structure supports continuous data processing.

Q16 A company maintains hierarchical employee records with multiple levels of management. Construct a concept map connecting Tree → Nodes → Levels → Traversal (DFS/BFS) → Searching → Efficiency. Analyze how traversal techniques affect performance and justify the most suitable approach for retrieving employee data.

Q17 A logistics company processes delivery requests across multiple cities. Construct a concept map linking Algorithm → Input Size → Time Complexity → Space Complexity → Scalability → Efficiency. Analyze how algorithm selection impacts system scalability and justify the best approach for handling large datasets.

Q18 A printer system manages print jobs submitted by multiple users. Construct a concept map connecting Queue → FIFO → Job Scheduling → Processing → Efficiency and Stack → LIFO → Undo Operations → Execution. Explain how each structure is applied and justify their suitability in this context.

Q19 A system processes hierarchical organizational data with multiple nested levels. Construct a concept map linking Recursion → Problem Decomposition → Base Case → Recursive Calls → Call Stack → Memory Usage. Analyze how recursion simplifies problem-solving and evaluate its impact on memory.

Q20 A video streaming application must balance speed and memory usage. Construct a concept map connecting Time Complexity → Space Complexity → Tradeoff → Algorithm Design → Performance Optimization. Analyze how improving one factor affects the other and justify the optimal balance.

Q21 A student attendance system records daily presence data. Construct a concept map linking Array → Fixed Size → Sequential Access → Linked List → Dynamic Allocation → Flexibility. Explain how each structure works and justify which is more suitable for dynamic data updates.

Q22 A delivery network connects multiple warehouses and delivery points. Construct a concept map connecting Graph → Nodes → Edges → Weighted Graph → Shortest Path → Efficiency. Analyze how graphs model real-world networks and justify their use in route optimization.

Q23 A digital archive stores millions of documents. Construct a concept map linking Searching → Linear Search → Binary Search → Sorted Data → Time Complexity → Performance. Analyze the impact of data organization on search efficiency and justify the best technique.

Q24 A payroll system calculates salaries, bonuses, and deductions. Construct a concept map connecting ADT → Functions → Data Abstraction → Implementation → Reusability → Maintainability. Explain how modular design improves system development and justify the use of ADT.

Q25 A program processes deeply nested expressions. Construct a concept map linking Recursion Depth → Call Stack → Memory Usage → Stack Overflow → Efficiency. Analyze the risks of deep recursion and evaluate alternative approaches.

Q26. A real-time gaming system requires fast processing and minimal delay. Construct a concept map connecting Algorithm Optimization → Time Complexity Reduction → Space Optimization → Performance → User Experience. Analyze how optimization improves system performance and justify techniques used

Q27. A payroll system processes employee salaries sequentially. Construct a concept map connecting Array → Sequential Access → Processing → Efficiency → Limitations. Explain how sequential processing works and justify its advantages.

Q28. A big data system processes millions of records. Construct a concept map connecting Performance → Time Complexity → Space Complexity → Throughput → Efficiency → Optimization. Analyze how performance metrics influence system design and justify optimization strategies.

Q29. A sorting system uses recursive algorithms to process data. Construct a concept map linking Recursion → Divide and Conquer → Subproblems → Call Stack → Time Complexity → Efficiency. Analyze how recursion improves problem-solving and evaluate its performance impact.

Q30. A retail system manages product inventory and sales records. Construct a concept map connecting Data Structure → Operations (Insert/Delete/Search/Update) → Time Complexity → Efficiency → Performance. Explain how each operation affects system performance and justify the importance of efficient operations.

Q31. A school management system organizes student records class-wise and section-wise. Construct a concept map linking Sequential Storage → Arrays → Hierarchical Storage → Trees → Data Access → Efficiency. Explain how each approach works and justify which method is more suitable for hierarchical data.

Q32. A real-time online examination system processes student responses, evaluates answers, and stores results instantly. Construct a concept map linking Data Flow → Processing → Data Structure Selection → Access

Speed → Performance. Analyze how the flow of data influences the choice of data structures and justify the most efficient design.

Q33. A search engine must return results quickly while handling massive datasets. Construct a concept map linking Speed → Memory Usage → Time Complexity → Space Complexity → Trade-Off → Optimization. Analyze the trade-offs and justify how balance is achieved.

Q34. A software development team builds reusable modules for different applications. Construct a concept map connecting ADT → Module Design → Interface → Implementation → Reusability → Maintainability. Explain how modular design improves software development and justify its importance.

Q35. A program processes nested expressions using recursion and may encounter deep nesting. Construct a concept map linking Recursion → Call Stack → Stack Overflow → Error Handling → Efficiency. Analyze potential issues and justify strategies to handle them.

Q36. A messaging application stores chats, user lists, and message threads. Construct a concept map connecting Array → Linked List → Hashing → Hybrid Structures → Efficiency. Explain how combining multiple data structures improves performance and justify their use in real-time communication systems.

Q37. A wearable health device processes sensor data with limited battery and memory. Construct a concept map linking Constraints → Memory Limit → Processing Power → Data Structure Choice → Algorithm Efficiency. Analyze how constraints affect system design and justify optimal solutions.

Q38. A video streaming service frequently reads and updates user watch history. Construct a concept map connecting Data Access Patterns → Sequential Access → Random Access → Data Structures → Performance. Explain how access patterns influence structure selection and justify the most suitable approach.

Q39. A stock trading system processes thousands of transactions per second. Construct a concept map connecting Real-Time Data → Queue → Priority Handling → Processing → Efficiency. Explain how data structures support real-time processing and justify their use.

Q40. A software development team builds reusable modules for different applications. Construct a concept map connecting ADT → Module Design → Interface → Implementation → Reusability → Maintainability. Explain how modular design improves software development and justify its importance.

7)

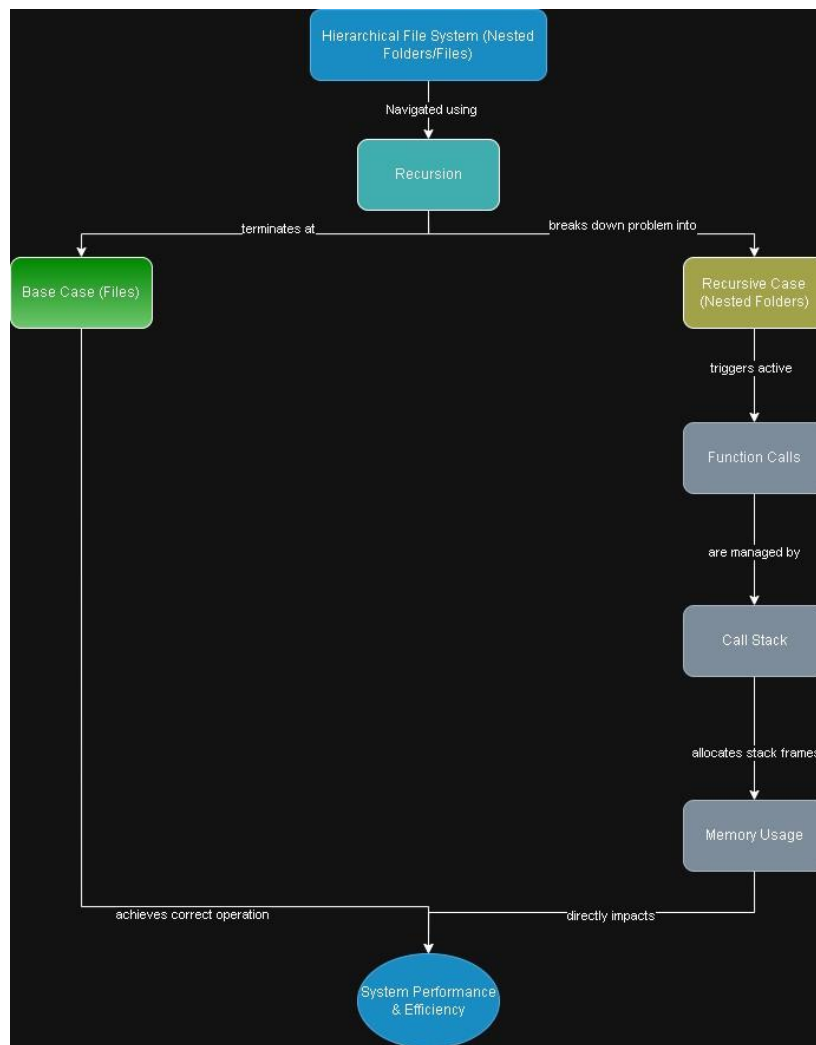


Figure 1: Concept Map for Recursion Operations in a Hierarchical File System

Technical Justification for Question: -

Structural Alignment to Hierarchical Data A file system is inherently non-linear and hierarchical, consisting of tree structures where directories contain nested sub-directories and individual files. Recursion is the most mathematically elegant and efficient approach for navigating this structure because a directory tree is a self-similar data structure—meaning any given sub-directory behaves exactly like a fully qualified root tree.

Problem Decomposition & Execution Flow The traversal process handles complex nesting by dividing the task into two definitive structural paths:

Recursive Case (Nested Folders): When the algorithm encounters a directory node, it pauses its immediate progression and dynamically spawns a new runtime instance of the function to evaluate that deeper branch.

Base Case (Files / Empty Folders): The execution path hits a structural termination point upon encountering an individual file or an empty directory. This triggers the base case, halting deeper traversal and allowing the active function to return back up the directory chain.

Memory Dynamics & Call Stack Mechanics At the runtime level, every recursive descent triggers an active function call that allocates a distinct *Stack Frame* onto the system's *Call Stack*. This frame caches local variables, parameter values, and execution return addresses. The memory usage scales linearly with the nesting depth, $O(d)$, where d represents the maximum depth of the directory tree. The execution strictly follows a Last-In, First-Out (LIFO) sequence, systematically unwinding frames as base cases validate, ensuring programmatic simplicity when parsing deeply nested structures.